

End-to-End Data Analytics Platform

Microsoft Fabric | Public Data | Spain Indicators

Complete technical specification defining environment setup, DDL scripts, notebook structures, Dataflow Gen2 configurations, pipeline orchestration, authentication, logging framework, DAX measures, and deployment strategy.

AUTHOR

Analytics Engineer

6 years experience

Microsoft Stack

STACK

Microsoft Fabric

Lakehouse | Warehouse

PySpark | Power BI

DATE

May 2026

Duration: 1-2 months

Version 1.1

1. Executive Summary

This document defines the technical specification for the End-to-End Data Analytics Platform. It is the implementation blueprint that translates the functional specification (Phase 3) into concrete engineering instructions: how the environment is provisioned, how objects are created, how notebooks are structured, how pipelines are orchestrated, how credentials are handled, and how the solution is deployed from development to production.

Where Phase 3 defined **WHAT** the system must produce (business rules, derived measures, source-to-target field mappings), this document defines **HOW** those outputs are engineered inside Microsoft Fabric. The two documents are complementary and must be read together: Phase 3 owns business intent; Phase 4 owns technical implementation.

1.1 Document Scope

This specification covers the technical design at the level of a professional engineering blueprint. It deliberately stops short of producing final production code. Code in its final form is authored during Phase 5 (Development), where it is iterated against the live Fabric environment and versioned in Git during Phase 6 (Publication).

In scope for this document	Out of scope (deferred to Phase 5 / 6)
Environment provisioning (Azure, Fabric workspaces, Lakehouse, Warehouse)	Production-ready PySpark source code (authored in Phase 5 against the live environment)
Complete DDL scripts for the Warehouse star schema	Final Power Query M code for Dataflows Gen2 (exported from Fabric in Phase 6)
Notebook structural specification (sections, functions, libraries, error handling)	Pipeline JSON/YAML export files (generated by Fabric, versioned in Phase 6)
Dataflow Gen2 step-by-step logical design (Power Query steps in prose)	Screenshots of the live Fabric workspace (captured during Phase 5)
Pipeline orchestration design (activities, parameters, dependencies, retry)	Final TMDL-format export files from the semantic model (generated in Phase 5)
Authentication architecture (cross-tenant Fabric to Azure SQL)	End-user documentation for the Power BI report
Logging framework implementation (DDL + helper function signatures)	Performance tuning results (measured after Phase 5)
Complete DAX measure definitions and formulas	Incremental loading patterns (explicitly out of project scope)
Deployment strategy (DEV to PROD via Fabric Deployment Pipelines)	CI/CD automation (explicitly out of project scope)
Validation queries and testing checklist per layer	Monitoring and alerting (explicitly out of project scope)

1.2 Why This Level of Detail

A technical specification written at the level of final code would be simultaneously premature and wasteful. **Premature**, because implementation inevitably reveals constraints that only emerge in the live environment: connector behavior, Direct Lake limitations, Dataflow Gen2 UI constraints, and Fabric-specific quirks that cannot be anticipated from documentation alone. **Wasteful**, because Phase 6 (GitHub Publication) versions the final code in detail with commit history, inline comments, and per-component README files — duplicating that effort here would produce two sources of truth that drift over time.

The appropriate analogy is an architectural blueprint for a building: the specification defines load-bearing walls, electrical paths, materials, and finishes, but does not prescribe the exact sequence in which the bricks are laid. This document is the blueprint. Phase 5 is the construction. Phase 6 is the as-built documentation.

***Guiding principle** — every technical decision in this document is stable enough to survive implementation unchanged in the majority of cases. Where implementation reveals a better path, the deviation is documented as a 'Phase 5 implementation decision' in the GitHub repository, preserving the original rationale for traceability.*

1.3 Document Structure

The document follows the natural sequence an engineer would traverse when implementing the solution from scratch: environment setup first, then standards and conventions, then objects (Warehouse DDL), then components (notebooks, dataflows, pipelines), then operational concerns (connections, implementation patterns, logging), then consumption (Power BI), then operationalization (deployment, testing, troubleshooting).

#	Section	Focus
1	Executive Summary	Scope, methodology, and document structure
2	Setup and Prerequisites	Azure and Fabric environment provisioning; real environment values; local development stack
3	Code Conventions and Standards	Notebook structure, naming, styling, logging pattern, Python package stack
4	Gold Layer DDL	Complete T-SQL for fact and dimension tables
5	Notebooks Specification	Structural design per notebook (Bronze, Silver, Gold)
6	Dataflows Gen2 Specification	Power Query step design for IPC and IPI
7	Data Pipelines Orchestration	Master, Bronze, Silver, Gold pipeline design
8	Connections and Authentication	Cross-tenant Fabric to Azure SQL configuration
9	Implementation Patterns: Azure SQL	Real-world findings from the IPI loader: connection strings, token packing, auto-pause, bulk insert performance, ODBC ARM64 setup, authentication strategy
10	Logging Framework Implementation	Log table DDL and helper function pattern
11	Power BI Technical Specification	Direct Lake setup, relationships, DAX catalog
12	Deployment Strategy	DEV to PROD via Fabric Deployment Pipelines
13	Testing and Validation Plan	Layer-by-layer verification queries and checklist
14	Troubleshooting Guide	Common errors and resolution patterns
15	Next Steps	Transition to Phase 5 Development
16	Changelog	Version history of this document

1.4 Audience

This document is written for a technical audience: data engineers, analytics engineers, and solution architects responsible for building or reviewing the solution. Business audiences should refer to Phase 3 (Functional Specification). Project governance and scope audiences should refer to Phase 0 (Project Brief).

1.5 Relationship to Other Phases

Upstream dependency	What this document inherits
Phase 0 — Project Brief	Scope boundaries, quality standards, capacity licensing strategy
Phase 1 — Requirements and Source Catalog	Five source definitions, access patterns, technical challenges per source
Phase 2 — Architecture Design	Medallion architecture, component mapping, object naming conventions, layered orchestration design
Phase 3 — Functional Specification	Source-to-target mappings, star schema fields, derived measure rules, data quality rules
Downstream consumer	What this document delivers
Phase 5 — Development	Complete implementation blueprint: DDL, notebook structure, pipeline activities, DAX catalog
Phase 6 — Publication	Source-of-truth for repository organization and component-level README structure

2. Setup and Prerequisites

This section defines everything that must exist before the first line of code is written in Phase 5. It covers two independent environments that must be provisioned and connected: the Azure subscription hosting the SQL Database (source system for IPI), and the Microsoft Fabric workspace hosting the Lakehouse, Warehouse, and all compute components.

Setup sequencing — Azure provisioning is performed **before** activating the Fabric trial. The Fabric trial is time-bounded and should be activated only when development is ready to begin. Every hour of the trial consumed on setup activities that could have been done beforehand is an hour lost to actual development.

2.1 Environment Overview

The platform uses a fully personal account stack across two cloud environments. Both environments live inside the same personal Entra ID tenant, simplifying authentication and identity governance.

Environment	Tenant	Identity	Purpose
Azure subscription	Personal Entra ID tenant	Microsoft Personal Account (tenant owner)	Hosts the Azure SQL Database that stores the IPI staging table
Microsoft Fabric workspace	Personal Entra ID tenant	Entra ID work-or-school user	Hosts the Lakehouse, Warehouse, notebooks, pipelines, semantic model, report

2.2 Azure Subscription and SQL Database

2.2.1 Azure Account Strategy

A personal Azure subscription is created using a personal Microsoft account. This decision is foundational and is documented in Phase 0 (Section 5). The relevant operational implications for technical setup are: (a) the subscription is independent of any employer infrastructure, (b) the author retains full administrative control, (c) costs are personally bounded via Azure budget alerts.

2.2.2 Azure SQL Database — Free Offer

The IPI staging database is provisioned under the Azure SQL Database free offer, which provides a serverless General Purpose database at no cost within its monthly allowance. The allowance is more than sufficient for portfolio usage: the IPI dataset is approximately 309,000 rows loaded once, and Fabric reads from it on demand during pipeline execution.

Free offer allowance	Value	Project usage estimate
Compute	100,000 vCore-seconds per month	Well under 5,000 vCore-seconds per month
Data storage	32 GB per database	Well under 100 MB
Backup storage	32 GB per database	Not actively used
Databases per subscription	Up to 10	One database required
Service tier	General Purpose serverless	Fits the portfolio use case
Maximum compute	4 vCores, auto-pause enabled	Compatible with occasional reads from Fabric
Duration	Lifetime of the subscription	No expiration risk

***Auto-pause billing protection** — at database creation, the option 'Auto-pause the database until next month' is selected. If monthly allowances are ever exhausted, the database pauses until the calendar resets and no charges are incurred. The alternative option ('Continue usage with additional charges') is explicitly not selected.*

2.2.3 Database Provisioning

STEP 1 — Create a resource group. Region: North Europe. Name: rg-spi-portfolio.

STEP 2 — Create a logical SQL Server. Authentication: **Microsoft Entra-only** (SQL authentication explicitly disabled). Entra admin set to the author's personal Microsoft account.

STEP 3 — Create the SQL Database (free offer). Workload environment: Development. Compute tier: Serverless. Free offer: **enabled**. Free limit behavior: **Auto-pause the database until next month**. Collation: SQL_Latin1_General_CP1_CI_AS.

STEP 4 — Configure firewall rules. Add the author's client IP as an allowed rule. Enable 'Allow Azure services and resources to access this server' so Fabric can reach the database.

2.2.4 IPI Data Load Into Azure SQL

The IPI source is originally an INE CSV (Source 2 in the functional spec). To simulate a transactional source system, the CSV is loaded once into a staging table in the Azure SQL Database via a local Python script (`ipi_bronze_load.py`) developed during Phase 4.5 (Local Python Validation). Fabric then reads from this staging table as if it were an enterprise transactional database.

Step	Tool	Action
Download	Python (requests)	Fetch the IPI CSV from the INE URL defined in the Phase 1 source catalog
Convert encoding	Python (pandas)	Convert from ISO-8859-15 to UTF-8 to ensure clean ingestion
Create staging table	Python (pyodbc + Entra ID)	DDL executed via authenticated connection (see Section 9)
Bulk insert	Python (pyodbc fast_executemany)	Load the UTF-8 dataframe into the staging table; ~309K rows in ~30 seconds
Validate	T-SQL	Row count check, distinct values on comunidades_y_ciudades_autonomas

Staging table DDL:

```
CREATE TABLE stg_ipi_raw (
    comunidades_y_ciudades_autonomas NVARCHAR(200) NOT NULL,
    destino_economico_de_los_bienes NVARCHAR(200) NOT NULL,
    tipo_de_dato NVARCHAR(100) NOT NULL,
    periodo NVARCHAR(10) NOT NULL,
    total NVARCHAR(50) NULL,
    _loaded_at DATETIME2(0) NOT NULL
    CONSTRAINT DF_stg_ipi_raw_loaded DEFAULT SYSUTCDATETIME()
);
GO

CREATE INDEX IX_stg_ipi_raw_ccaa
ON stg_ipi_raw (comunidades_y_ciudades_autonomas);
GO
```

Why NVARCHAR for all columns — the staging table mirrors the CSV exactly: every column is stored as a string. This protects against unexpected source format changes (stray non-numeric values in total, locale-specific decimal separators). Type casting happens in the Fabric Silver layer where it can be logged and validated.

Why a Python loader instead of bcp or Azure Data Studio import — the loader is reusable, version-controlled in the repository, demonstrates Entra-only authentication patterns, and surfaces real implementation findings

documented in Section 9. The loader is a portfolio asset in itself, not just a setup utility.

2.3 Microsoft Fabric Environment

2.3.1 Capacity and License

Fabric development uses the Fabric Trial Capacity activated within the personal tenant. The trial grants access to all Fabric workloads with up to 64 Capacity Units and 1 TB of OneLake storage. The trial is bounded in time; Section 2.5 details the time-management strategy. Post-trial, the workspace migrates to F2 PAYG (paused by default) per the Phase 0 capacity strategy.

Attribute	Value
Capacity type	Fabric Trial (F64 equivalent) during development; F2 PAYG (paused by default) post-trial
Duration (trial)	60 days standard (extendable by 60 days when less than 7 days remain)
Workloads included	All (Data Engineering, Data Factory, Data Warehouse, Power BI, Real-Time Analytics)
OneLake storage	1 TB (trial); fits well within F2 limits post-migration
Identity	Personal Entra ID work-or-school user (gaston@gastonballoiraoutlook.onmicrosoft.com)
Cost (post-trial)	\$0 paused; ~\$0.36/hour active; estimated \$5–15/month under typical usage

2.3.2 Workspace Topology

Workspace	Name	Role	Data scope
DEV	spi-spain-indicators-dev	Active development and iteration	Last 24 periods per source
PROD	spi-spain-indicators-prod	Final portfolio deployment and screenshot evidence	Full historical dataset per source

2.3.3 Core Fabric Items

Item type	Name	Purpose
Lakehouse	spi_lakehouse	Stores Bronze and Silver Delta tables and the logging table
Warehouse	spi_warehouse	Hosts the Gold star schema (1 fact + 5 dimensions)
Semantic Model	spi_spain_indicators	Direct Lake semantic model built on spi_warehouse
Power BI Report	spi_spain_indicators	Six-page report per Phase 3 specification
Notebooks, Pipelines, Dataflows	Per Phase 2 naming conventions	Created during Phase 5 as implementation progresses

2.3.4 Why Warehouse for Gold, Not Lakehouse SQL Endpoint

- **Portfolio breadth.** Using both Lakehouse and Warehouse demonstrates familiarity with both Fabric compute engines and their respective use cases.
- **Star schema appropriateness.** The Warehouse supports T-SQL DDL with primary keys, foreign keys, and constraints (declared but non-enforced) that are natural expressions of a dimensional model.
- **Enterprise pattern alignment.** The standard pattern of 'Lakehouse for raw and cleaned data, Warehouse for dimensional model' mirrors the Bronze-Silver in Lakehouse and Gold in Warehouse split used here.

2.4 Local Development Stack

Several activities during Phase 5 (and the entire Phase 4.5 local validation work) require a local development environment. This section documents the actual stack used by the author, demonstrating that the project is reproducible in any modern environment.

Component	Specification	Notes
Operating system	macOS Tahoe 26.4	ARM64 architecture; specific ODBC driver requirements documented in Section 9
Hardware	MacBook Air M3, 16 GB RAM	Apple Silicon; sufficient for local data validation up to ~1 GB datasets
Python	3.12.13 (via Homebrew)	Project dependencies managed in an isolated virtualenv (.venv/)
IDE	Visual Studio Code 1.118	Python extension suite + MSSQL extension
Database client	VS Code MSSQL extension	Replaces Azure Data Studio (retired by Microsoft Feb 2026)
Azure CLI	Latest stable	For local credential caching via az login (avoids browser MFA on every script run)
Version control	Git 2.50 + GitHub	Repository hosted at https://github.com/Gaston-080387/spi-spain-indicators
Browser	Chrome / Edge	For Fabric Service portal and Azure Portal

Justification — *the entire stack is free, cross-platform, and professionally adopted. There is no proprietary or paid tool in the local development chain. This demonstrates that the project can be reproduced by any developer on any modern environment without licensing barriers.*

2.5 Trial Time Management Strategy

Activity	Trial active?	When
Create personal Entra ID tenant + Azure subscription	No	Pre-Phase 5
Provision Azure SQL Database and load IPI CSV	No	Phase 4.5 (Local Python Validation)
Draft all Phase 4 content (DDL, notebook structure, DAX)	No	This document
Activate Fabric trial	Day 1	Phase 5 kickoff
Create DEV workspace, Lakehouse, Warehouse	Yes	Phase 5 Day 1-2
Implement and test five source pipelines	Yes	Phase 5 Week 1-5
Deploy DEV to PROD, capture screenshots	Yes	Phase 5 final week
Migrate PROD to F2 PAYG (paused)	Final	Before day 53 of trial
Publish artifacts to GitHub	Optional	Phase 6

2.6 Environment Reference

This subsection documents the actual environment values for the author's deployment. Other engineers reproducing the project will substitute these with their own values.

2.6.1 Tenant Information

Attribute	Value
Tenant Name	Directorio predeterminado
Tenant ID	8052fb97-6ef5-4dbd-a2e4-5cc5890ac1ae
Primary Domain	gastonbaloiroutlook.onmicrosoft.com
License	Microsoft Entra ID Free

2.6.2 Identities

Role	Identity	Used For
Owner (Azure)	gaston.baloira@outlook.com (Microsoft Personal Account)	Owning the personal Azure subscription and tenant administration
Fabric / SQL technical user	gaston@gastonbaloiroutlook.onmicrosoft.com (Entra ID work-or-school)	Logging into Fabric, owning workspaces, connecting to Azure SQL via Entra-only auth

2.6.3 Azure Subscription

Attribute	Value
Type	Pay-As-You-Go (post free trial conversion)
Region	North Europe
Resource Group	rg-spi-portfolio
Budget Alert	\$25 USD/month with 50% / 80% / 100% thresholds

2.6.4 Azure SQL (Source 2 — IPI)

Attribute	Value
Server	spi-sqlserver-gb.database.windows.net
Database	spi_staging
Tier	General Purpose Serverless (Free Offer applied)
Authentication	Microsoft Entra-only (no SQL auth)
Free Quota	32 GB storage / 100K vCore-seconds per month
Auto-pause	Enabled (paused after ~1h inactivity, ~30s resume)

2.6.5 GitHub Repository

Attribute	Value
URL	https://github.com/Gaston-080387/spi-spain-indicators
Visibility	Public
License	MIT

Sensitive values — the tenant ID and resource names above are non-secret identifiers safe to publish. Authentication is Entra-only (no passwords or connection strings stored in the repository). The repository's .gitignore excludes any credential cache files generated by the Azure CLI or VS Code extensions.

3. Code Conventions and Standards

Consistency across all Fabric components is essential for a portfolio project. A reviewer should be able to open any notebook, pipeline, or dataflow and immediately recognize the same structure, naming, and logging pattern. This section defines those conventions as cross-cutting rules applied throughout Phase 5.

3.1 Naming Conventions

Naming inherits directly from Phase 2 (Architecture Design, section 8) and is repeated here for self-contained reference.

Object type	Pattern	Example
Global prefix	spi_	spi_lakehouse
Tables and columns	snake_case, lowercase	indicator_value, region_name
Bronze tables	spi_bronze__raw	spi_bronze_ipc_raw
Silver tables	spi_silver__cleaned	spi_silver_ipc_cleaned
Gold fact	spi_fact_	spi_fact_indicators
Gold dimensions	spi_dim_	spi_dim_region
Notebooks	spi_nb__	spi_nb_bronze_energy
Pipelines	spi_pl_	spi_pl_master, spi_pl_bronze
Dataflows Gen2	spi_df__	spi_df_silver_ipc
Log table	Operational prefix	spi_log_pipeline_execution
Bronze metadata columns	Underscore prefix	_ingestion_timestamp, _source_file
Workspaces	kebab-case with environment suffix	spi-spain-indicators-dev

3.2 Notebook Structure

Section	Purpose
1. Header	Metadata markdown cell with notebook name, layer, source, owner, purpose, last-updated date
2. Imports	All library imports grouped: standard library first, third-party second, Fabric/PySpark utilities last
3. Parameters	Parameterized cell (tagged 'parameters' for pipeline injection). Defines inputs such as workspace, lakehouse name, date ranges, source URLs
4. Functions	All reusable logic defined as pure functions with docstrings. No side effects inside function definitions
5. Main	Orchestration block that calls functions in order, writes output, and updates the logging table. Wrapped in try/except

Rationale — separating functions from main enables unit-style testing of individual transformations without running the full ingestion. It also makes the main block short and readable: a reviewer can understand the notebook's flow in under a minute.

3.3 PySpark Style

Rule	Rationale
Use snake_case for all Python variables, function names, and column names	Consistency with the rest of the project and PEP 8
Prefer DataFrame API over SQL strings for transformations	Easier to read, compose, and test in isolation
Explicit schema definitions when reading semi-structured data (JSON, XLSX)	Avoids silent schema drift when source changes
Short, single-purpose functions (20-30 lines maximum)	Testability and readability
Docstrings on every function (1-2 lines describing input, output, side effect)	Enables notebook-to-doc extraction in Phase 6
No magic numbers: constants defined at the top of the notebook	Makes parameters visible and changeable in one place
Type hints on function signatures when reasonably possible	Documents intent and helps IDE tooling

3.4 Logging Pattern

Every notebook and every pipeline must write to the centralized log table `spi_log_pipeline_execution` (specified in Section 10). The pattern is uniform across components:

- **Start:** insert a row with `status = 'running'` and `start_time = now`.
- **Work:** execute the notebook's main logic.
- **End (success):** update the row with `status = 'success'`, `end_time = now`, `rows_processed = N`.
- **End (failure):** update the row with `status = 'failed'`, `end_time = now`, `error_message = <exception>`.

A shared helper module exposes three functions: `log_start()`, `log_end_success()`, `log_end_failure()`. Every notebook imports and uses these helpers rather than writing raw logging logic.

3.5 Error Handling Philosophy

Principle	Application
Catch and log errors at the notebook boundary	Each notebook wraps its main block in try/except, logs the failure, and re-raises so the pipeline captures it
Do not swallow errors silently inside functions	Functions either succeed or raise; no partial-state fallbacks
Data quality violations produce warnings, not exceptions	Rows failing DQ rules are rejected (or preserved with warning per Phase 3 section 6.1); the notebook continues
Transient errors (network, API timeouts) use bounded retry	Only where documented (REE API); file/DB errors do not retry
Error messages must be actionable	Include the source, the step, and what was expected vs. received

3.6 Comments and Documentation

The balance is: **documentation where code cannot speak for itself**.

- Write comments for *why*, not *what*. The code shows what; comments explain decisions, tradeoffs, or source quirks.
- Use markdown cells in notebooks to separate major stages and explain business rules or source-specific gotchas.

- Every notebook has a top markdown cell summarizing purpose and dependencies. Every pipeline has a description field filled in.
- Dead code is deleted, not commented out. Git preserves history.

3.7 Python Package Stack

The local Python environment used for Phase 4.5 (Local Python Validation) and as the foundation for the Fabric notebook environment is documented below. Versions reflect what was actually used and validated by the author; Phase 5 may update them based on Fabric's installed runtime.

Package	Version	Use
pandas	3.0.2	DataFrames, tabular manipulation, in-memory transformations
openpyxl	3.1.5	Modern XLSX reading (Source 5: Tax Revenue)
xlrd	2.0.2	Legacy XLS reading with merged-cell support (Source 4: Construction)
requests	2.33.1	HTTP requests with retry support (Source 3: Energy REE API)
pyarrow	24.0.0	Pandas backend, Parquet, Delta Lake interop
pyodbc	Latest stable	Azure SQL connection (added in Phase 4.5 — see Section 9)
azure-identity	Latest stable	Microsoft Entra ID authentication for Azure SQL (added in Phase 4.5 — see Section 9)

Rationale for explicit version pinning — *documenting the exact versions used during validation reduces future reproduction friction. When a reviewer or a future Claude instance runs `pip install -r requirements.txt`, they get the same library behavior. Floating-version installs (no pin) defeat reproducibility.*

Distinction between local and Fabric environments — *Fabric notebooks run on a managed Spark runtime with a pre-installed set of libraries. The local stack above is for Phase 4.5 validation work and for any auxiliary scripts versioned in the repository (such as `ipi_bronze_load.py`). The Phase 5 Fabric notebooks rely on Fabric's bundled libraries except where additional packages are installed via Fabric environment configuration.*

4. Gold Layer DDL

This section defines the complete T-SQL DDL for the six Warehouse tables that implement the star schema. The DDL is stable and can be executed as-is in Phase 5.

Note on Fabric Warehouse constraints — Fabric Warehouse supports primary key and foreign key declarations, but they are **NOT ENFORCED** (informational only). They are declared here because (a) they document the intended model for reviewers, (b) they enable the Power BI semantic model to detect relationships automatically, (c) they align with industry-standard data modeling practices.

4.1 Schema Organization

All Gold objects live in the dbo schema of spi_warehouse. A dedicated schema (e.g. gold) is not used because Fabric Warehouse schema management adds complexity without benefit at this scale.

4.2 Fact Table — spi_fact_indicators

```
CREATE TABLE dbo.spi_fact_indicators (  
    indicator_key    INT            NOT NULL,  
    region_key       INT            NOT NULL,  
    calendar_key     INT            NOT NULL,  
    measure_key      INT            NOT NULL,  
    source_key       INT            NOT NULL,  
    indicator_value  DECIMAL(18,4) NULL,  
    CONSTRAINT PK_fact_indicators PRIMARY KEY NONCLUSTERED (  
        indicator_key, region_key, calendar_key, measure_key, source_key  
    ) NOT ENFORCED,  
    CONSTRAINT FK_fact_indicator  
        FOREIGN KEY (indicator_key)  
        REFERENCES dbo.spi_dim_indicator (indicator_key) NOT ENFORCED,  
    CONSTRAINT FK_fact_region  
        FOREIGN KEY (region_key)  
        REFERENCES dbo.spi_dim_region (region_key) NOT ENFORCED,  
    CONSTRAINT FK_fact_calendar  
        FOREIGN KEY (calendar_key)  
        REFERENCES dbo.spi_dim_calendar (calendar_key) NOT ENFORCED,  
    CONSTRAINT FK_fact_measure  
        FOREIGN KEY (measure_key)  
        REFERENCES dbo.spi_dim_measure (measure_key) NOT ENFORCED,  
    CONSTRAINT FK_fact_source  
        FOREIGN KEY (source_key)  
        REFERENCES dbo.spi_dim_source (source_key) NOT ENFORCED  
);
```

Column	Type	Notes
indicator_key	INT NOT NULL	FK to spi_dim_indicator
region_key	INT NOT NULL	FK to spi_dim_region
calendar_key	INT NOT NULL	FK to spi_dim_calendar, format YYYYMM
measure_key	INT NOT NULL	FK to spi_dim_measure (IDX, YoY, MoM, YTD, QTD)
source_key	INT NOT NULL	FK to spi_dim_source
indicator_value	DECIMAL(18,4) NULL	NULL allowed for derived measures per Phase 3 business rules

4.3 Dimension — spi_dim_indicator

```
CREATE TABLE dbo.spi_dim_indicator (
    indicator_key      INT          NOT NULL,
    indicator_name     NVARCHAR(200) NOT NULL,
    indicator_category NVARCHAR(200) NOT NULL,
    domain             NVARCHAR(100) NOT NULL,
    unit_of_measure    NVARCHAR(100) NOT NULL,
    frequency          NVARCHAR(50)  NOT NULL,
    CONSTRAINT PK_dim_indicator PRIMARY KEY NONCLUSTERED (indicator_key) NOT ENFORCED
);
```

4.4 Dimension — spi_dim_region

```
CREATE TABLE dbo.spi_dim_region (
    region_key      INT          NOT NULL,
    region_code     NVARCHAR(3)  NOT NULL,
    region_name     NVARCHAR(50) NOT NULL,
    region_type     NVARCHAR(50) NOT NULL,
    CONSTRAINT PK_dim_region PRIMARY KEY NONCLUSTERED (region_key) NOT ENFORCED
);
```

Seed data (three rows, inserted once in Phase 5):

region_key	region_code	region_name	region_type
1	NAC	Nacional	Aggregated National
2	CAT	Cataluña	Autonomous Community
3	MAD	Madrid	Autonomous Community

4.5 Dimension — spi_dim_calendar

```
CREATE TABLE dbo.spi_dim_calendar (
    calendar_key      INT          NOT NULL,
    year              INT          NOT NULL,
    month             INT          NOT NULL,
    month_name        NVARCHAR(20) NOT NULL,
    quarter           INT          NOT NULL,
    year_month        NVARCHAR(7)  NOT NULL,
    is_current_year    BIT          NOT NULL,
    CONSTRAINT PK_dim_calendar PRIMARY KEY NONCLUSTERED (calendar_key) NOT ENFORCED
);
```

calendar_key is a smart surrogate: the integer YYYYMM. This enables direct arithmetic filtering in T-SQL and DAX without joins. The dimension is programmatically populated covering 2000 through the current year (approximately 324 rows).

Note on schema alignment — this dimension does not contain a date column. The grain is monthly, so individual days are not meaningful. Phase 3 section 4.4 already defines this model; Phase 4 implements it verbatim.

4.6 Dimension — spi_dim_measure

```
CREATE TABLE dbo.spi_dim_measure (
    measure_key      INT          NOT NULL,
    measure_code     NVARCHAR(10) NOT NULL,
    measure_name     NVARCHAR(100) NOT NULL,
    measure_description NVARCHAR(500) NOT NULL,
    CONSTRAINT PK_dim_measure PRIMARY KEY NONCLUSTERED (measure_key) NOT ENFORCED
);
```

Seed data (five rows):

key	code	name	description
1	IDX	Index / Original Value	Original value as provided by the source
2	YoY	Year-over-Year Rate	% change vs. same month of previous year
3	MoM	Month-over-Month Rate	% change vs. immediately preceding month
4	YTD	Year-to-Date Cumulative	Cumulative % change from Jan to current month vs. same period of previous year
5	QTD	Quarter-to-Date Cumulative	Cumulative % change from quarter start vs. same period of previous year

4.7 Dimension — spi_dim_source

```
CREATE TABLE dbo.spi_dim_source (
    source_key          INT          NOT NULL,
    source_code         NVARCHAR(20) NOT NULL,
    source_name         NVARCHAR(200) NOT NULL,
    source_institution  NVARCHAR(100) NOT NULL,
    source_domain       NVARCHAR(100) NOT NULL,
    source_url          NVARCHAR(500) NULL,
    update_frequency    NVARCHAR(50) NOT NULL,
    CONSTRAINT PK_dim_source PRIMARY KEY NONCLUSTERED (source_key) NOT ENFORCED
);
```

Seed data (five rows):

key	code	name	institution	domain
1	IPC	Consumer Price Index	INE	Consumer Prices
2	IPI	Industrial Production Index	INE	Industrial Production
3	ENERGY	Electricity Demand	REE (Redeia)	Energy
4	CONSTRUCTION	Public Construction Tenders	MITMA	Construction
5	TAX	Tax Revenue by Delegations	AEAT	Tax Revenue

4.8 Execution Order

The DDL must be executed in dependency order: dimensions first (they have no FKs), then the fact table (which references all dimensions). The seed inserts for fixed dimensions (region, measure, source, calendar) run immediately after each dimension is created. The indicator dimension is populated during Phase 5 by the Gold notebook as sources are processed.

5. Notebooks Specification

This section defines the structural design of each notebook without prescribing final code. Every notebook follows the conventions of Section 3 (header, imports, parameters, functions, main) and uses the logging helpers of Section 10.

5.1 Notebook Inventory

Notebook	Layer	Source	Complexity
spi_nb_bronze_energy	Bronze	REST API (REE)	Medium-High
spi_nb_bronze_construction	Bronze	XLS Legacy (MITMA)	High
spi_nb_bronze_tax	Bronze	XLSX Complex (AEAT)	High
spi_nb_silver_energy	Silver	Bronze Energy	Medium
spi_nb_silver_construction	Silver	Bronze Construction	High
spi_nb_silver_tax	Silver	Bronze Tax	High
spi_nb_gold_load	Gold	All five Silver tables	High

IPC and IPI do not have notebooks: they are handled end-to-end by Dataflows Gen2 (Section 6).

5.2 Bronze Notebook — Energy (REE API)

Purpose. Ingest Spain's electricity demand evolution from the REE public REST API into `spi_bronze_energy_raw` (Delta table in Lakehouse).

Inputs.

Parameter	Description
start_year	First year to fetch (default: 2014)
end_year	Last year to fetch (default: current year)
geo_entities	List of {geo_limit, geo_ids, geo_name} tuples for Nacional, Cataluña, Madrid
api_base_url	REE API base URL

Output. Delta table `spi_bronze_energy_raw` with all fields as string + metadata columns (`_ingestion_timestamp`, `_source_file`).

Key libraries. `requests` (HTTP), `json` (parsing), `pyspark.sql` (DataFrame construction and Delta write), `time` (backoff sleep).

Functions (high-level inventory).

Function	Responsibility
<code>build_request_url()</code>	Construct the API URL given year, geo_limit, geo_ids
<code>fetch_with_retry()</code>	Execute HTTP GET with 30s timeout, 3 retries with exponential backoff (2s, 4s, 8s), retry only on 429/500/502/503/504
<code>parse_response()</code>	Navigate the JSONAPI structure <code>included[]</code> → <code>attributes.values[]</code> and return a list of rows
<code>build_dataframe()</code>	Convert the list of rows into a PySpark DataFrame with explicit schema
<code>write_bronze()</code>	Append to Delta table with schema merge disabled (schema stays fixed)

Error handling. Transient HTTP errors trigger the retry policy. Permanent errors (400, 401, 404) fail immediately with a descriptive message. JSON parse failures include the offending response snippet in the error message. All errors propagate to the main block, which logs failure to `spi_log_pipeline_execution` and re-raises.

Volume notes. Three API calls per year (one per geo entity), approximately 12 years of data → ~36 calls per execution. Total rows in Bronze: ~500 per entity-year × 3 entities × 12 years = ~18,000 rows.

5.3 Bronze Notebook — Construction (XLS Legacy)

Purpose. Download four legacy XLS files from the MITMA statistical bulletin and consolidate them into `spi_bronze_construction_raw`.

Key libraries. `requests` (download), `xlrd` (read legacy .xls), `pandas` (intermediate parsing of merged cells), `pyspark.sql` (Delta write).

Functions (high-level inventory).

Function	Responsibility
<code>download_file()</code>	Fetch XLS from URL, save to temporary storage
<code>parse_xls_sheet()</code>	Open with <code>xlrd</code> , handle merged cells, extract multi-row header, return <code>DataFrame</code>
<code>derive_metadata()</code>	Add <code>contracting_agent</code> and <code>region_name</code> columns from file-name mapping
<code>consolidate()</code>	Concatenate the four <code>DataFrames</code> into one
<code>write_bronze()</code>	Write consolidated <code>DataFrame</code> to Delta

Complexity notes. Merged cells are detected via `xlrd`'s `merged_cells` attribute and expanded by forward-filling values. Multi-row headers are flattened into a single header row by concatenation. Subtotal rows (identified by label patterns like 'TOTAL', 'SUBTOTAL') are left in Bronze — they are removed in Silver.

5.4 Bronze Notebook — Tax Revenue (XLSX Complex)

Purpose. Extract the `datos_delegaciones` sheet from the AEAT multi-sheet workbook and write it to `spi_bronze_tax_raw`.

Key libraries. `requests` (download), `openpyxl` (read XLSX), `pandas` (sheet extraction), `pyspark.sql` (Delta write).

Functions (high-level inventory).

Function	Responsibility
<code>download_workbook()</code>	Fetch XLSX from URL
<code>extract_target_sheet()</code>	Open with <code>openpyxl</code> , read only <code>datos_delegaciones</code> , return raw values
<code>to_dataframe()</code>	Build a <code>DataFrame</code> with header from row 1, all cells as strings
<code>write_bronze()</code>	Write <code>DataFrame</code> to Delta

Complexity notes. The workbook is ~6 MB but only one sheet is in scope. Opening with `read_only=True` and targeted sheet selection keeps memory usage low.

5.5 Silver Notebook — Energy

Purpose. Transform `spi_bronze_energy_raw` into `spi_silver_energy_cleaned`: flatten, deduplicate, type-enforce, filter per Phase 3 rules.

Key transformations.

- Filter: `indicator_title = 'Demanda real'` only.
- Cast value to decimal; cast datetime → derive year, month.
- Map `geo_name` → `region_name` (Nacional, Cataluña, Madrid).
- Deduplicate by (`region_name`, `year`, `month`), keeping the latest by `_ingestion_timestamp`.
- Drop null / zero values per DQ-08 (with warning logged).

5.6 Silver Notebook — Construction**Key transformations.**

- Remove subtotal rows (label-based filter).
- Normalize column names to `snake_case`.
- Parse `period_raw` → year, month.
- Cast `indicator_value_raw` → decimal.
- Preserve `contracting_agent` for downstream concatenation in Gold.
- Normalize `region_name` to the three canonical values.

5.7 Silver Notebook — Tax Revenue**Key transformations.**

- **Unpivot:** geographic columns become rows. Retain only Total (Nacional), Madrid, Cataluña columns.
- Parse period info → year, month.
- Cast value to decimal.
- Map tax concept row labels to `indicator_category`.

5.8 Gold Notebook — Load

Purpose. Load all five Silver tables into the Gold star schema in `spi_warehouse`. Calculate the four derived measures (YoY, MoM, YTD, QTD) per Phase 3 business rules. Populate `spi_dim_indicator` as new indicators are encountered.

Functions (high-level inventory).

Function	Responsibility
read_silver_tables()	Load all five Silver tables into DataFrames
union_silver()	Concatenate into a unified DataFrame with a source_code column
upsert_indicator_dim()	For each unique (indicator_category, domain), insert into spi_dim_indicator if not present
lookup_dimension_keys()	Join to each dimension to retrieve indicator_key, region_key, calendar_key, source_key
compute_yoy()	Window function partitioned by (indicator_key, region_key), lag 12 months
compute_mom()	Window function partitioned by (indicator_key, region_key), lag 1 month
compute_ytd()	Cumulative sum from January of same year vs same period previous year
compute_qtd()	Cumulative sum from quarter start vs same period previous year
union_measures()	Concatenate IDX rows with derived measure rows, each with appropriate measure_key
write_warehouse()	Truncate and reload spi_fact_indicators

Volume estimate. ~60,000 Silver rows × 5 measures = ~300,000 fact rows after expansion. Well within Warehouse capacity.

6. Dataflows Gen2 Specification

Dataflows Gen2 handle the two simplest sources (IPC, IPI) where Spark overhead is not justified. This section describes the Power Query steps in prose order; final M code is produced in Phase 5 and exported in Phase 6.

6.1 Dataflow Inventory

Dataflow	Layer	Source → Target
spi_df_silver_ipc	Silver	spi_bronze_ipc_raw → spi_silver_ipc_cleaned
spi_df_silver_ipi	Silver	spi_bronze_ipi_raw → spi_silver_ipi_cleaned
spi_df_gold_ipc	Gold	spi_silver_ipc_cleaned → rows in spi_fact_indicators
spi_df_gold_ipi	Gold	spi_silver_ipi_cleaned → rows in spi_fact_indicators
spi_df_gold_dimensions	Gold	Seed inserts for spi_dim_region, spi_dim_measure, spi_dim_source, and generation of spi_dim_calendar

6.2 Dataflow — spi_df_silver_ipc (and IPI, analogous)

Source. Lakehouse table spi_bronze_ipc_raw.

Steps (ordered).

1. **Connect** to the Bronze Delta table via Lakehouse connector.
2. **Filter rows** where tipo_de_dato = 'Indice' (discard pre-calculated variations).
3. **Filter rows** where comunidades_y_ciudades_autonomas is in the list: 'Nacional', 'Cataluña', 'Madrid'.
4. **Rename column** comunidades_y_ciudades_autonomas → region_name.
5. **Rename column** grupos_ecoicop → indicator_category.
6. **Split column** periodo (format YYYYMMNN) into year and month.
7. **Rename column** total → indicator_value.
8. **Change column type** of indicator_value to decimal.
9. **Remove columns:** drop Bronze metadata.
10. **Destination:** write to Lakehouse as spi_silver_ipc_cleaned, replace mode.

6.3 Dataflow — spi_df_gold_ipc (and IPI, analogous)

Steps (ordered).

1. **Connect** to the Silver Delta table.
2. **Lookup** indicator_key from spi_dim_indicator.
3. **Lookup** region_key from spi_dim_region.
4. **Compute** calendar_key as year * 100 + month.
5. **Add constant columns:** measure_key = 1 (IDX), source_key per source.
6. **Select columns:** the five keys plus indicator_value.
7. **Destination:** append rows to Warehouse table spi_fact_indicators.

Why IDX only here — YoY, MoM, YTD, QTD require window functions over the full fact table. They are computed by the Gold notebook (spi_nb_gold_load, Section 5.8) after all Silver tables have loaded their IDX rows.

7. Data Pipelines Orchestration

Pipelines implement the layered orchestration defined in Phase 2: a master pipeline invokes three layer pipelines (Bronze, Silver, Gold) which in turn invoke the appropriate notebooks and dataflows.

7.1 Pipeline Inventory

Pipeline	Role
spi_pl_master	Top-level orchestrator; runs Bronze → Silver → Gold in sequence
spi_pl_bronze	Ingests all five sources into Bronze
spi_pl_silver	Transforms Bronze → Silver for all sources
spi_pl_gold	Loads Silver → Gold Warehouse for all sources

7.2 Pipeline — spi_pl_master

Attribute	Value
Activities	Three 'Invoke Pipeline' activities chained with 'On Success' dependencies
Activity 1	Invoke spi_pl_bronze
Activity 2	Invoke spi_pl_silver — runs only if Bronze succeeded
Activity 3	Invoke spi_pl_gold — runs only if Silver succeeded
Parameters	environment (dev / prod), data_scope (last_24_periods / full_history)
Logging	Writes a single master log row at start and updates at end

7.3 Pipeline — spi_pl_bronze

Activity	Type	Runs
1	Copy Activity	IPC: INE URL (HTTP) → spi_bronze_ipc_raw
2	Copy Activity	IPI: Azure SQL stg_ipi_raw → spi_bronze_ipi_raw
3	Notebook Activity	Runs spi_nb_bronze_energy
4	Notebook Activity	Runs spi_nb_bronze_construction
5	Notebook Activity	Runs spi_nb_bronze_tax

Dependencies. All five activities run in parallel. Each is independent.

On-error behavior. 'Continue on error' at pipeline level. Each activity logs its own success/failure.

Retry. Only the Energy Notebook activity has pipeline-level retry enabled: 3 attempts with 30-second interval.

7.4 Pipeline — spi_pl_silver

Activity	Type	Runs
1	Dataflow Gen2	spi_df_silver_ipc
2	Dataflow Gen2	spi_df_silver_ipi
3	Notebook Activity	spi_nb_silver_energy
4	Notebook Activity	spi_nb_silver_construction
5	Notebook Activity	spi_nb_silver_tax

7.5 Pipeline — spi_pl_gold

Activity	Type	Runs
1	Dataflow Gen2	spi_df_gold_dimensions
2	Dataflow Gen2	spi_df_gold_ipc
3	Dataflow Gen2	spi_df_gold_ipi
4	Notebook Activity	spi_nb_gold_load (Energy, Construction, Tax + derived measures for all five sources)

Dependencies. Activity 1 must complete before any other Gold activity. Activities 2, 3 run in parallel after Activity 1. Activity 4 waits for Activities 2 and 3 to complete.

7.6 Pipeline Parameters

Parameter	Used by	Values
environment	All pipelines	dev, prod
data_scope	All pipelines	last_24_periods, full_history
run_id	All pipelines	GUID generated at master start, propagated to all child activities for log correlation

8. Connections and Authentication

This section specifies how Fabric connects to the Azure SQL Database. Both environments live inside the same personal Entra ID tenant, so authentication uses Entra ID single sign-on consistently across the project.

8.1 Same-Tenant Architecture

Fabric workspace identity (gaston@gastonbaloir outlook.onmicrosoft.com) and the Azure SQL Database both live in the personal Entra ID tenant gastonbaloir outlook.onmicrosoft.com. This enables Entra-only authentication without SQL credentials, simplifying the security posture.

8.2 Authentication Method — Microsoft Entra ID

The Azure SQL Database is configured with **Microsoft Entra-only authentication** (SQL authentication explicitly disabled). Fabric's Data Pipeline Copy Activity authenticates using the Fabric workspace identity, which is granted explicit permissions on the database.

Component	Configuration
Azure SQL auth mode	Microsoft Entra-only (no SQL authentication)
Fabric connection auth	Microsoft Entra ID (user identity or service principal)
Database user	Created from the Entra ID identity, granted SELECT on stg_ipi_raw
Network	Public endpoint with firewall: 'Allow Azure services and resources to access this server' enabled

8.3 Granting Database Access to the Fabric Identity

The following T-SQL is executed once in the Azure SQL Database as the Entra admin to grant Fabric read access to the staging table:

```
-- Run as Entra admin in spi_staging database
CREATE USER [gaston@gastonbaloir outlook.onmicrosoft.com]
FROM EXTERNAL PROVIDER;

GRANT SELECT ON dbo.stg_ipi_raw
TO [gaston@gastonbaloir outlook.onmicrosoft.com];
```

In production with a service principal, the syntax is identical except the user is created from the SP's display name.

8.4 Fabric Connection Object

Field	Value
Connection name	conn_azure_sql_ipi_source
Server	spi-sqlserver-gb.database.windows.net
Database	spi_staging
Authentication	Microsoft Entra ID
Privacy level	Organizational
Encryption	Required (TLS 1.2+)

8.5 Why Entra-Only (and Not SQL Authentication)

- **No password storage:** nothing to leak, rotate, or forget.
- **Audit trail:** every connection is tied to a known identity in Entra ID.
- **Modern standard:** Microsoft's recommended approach for new Azure SQL deployments.
- **MFA-compatible:** the user identity inherits MFA policies from the tenant.
- **Fabric-native:** the Fabric Connection picker offers Entra ID as a first-class option.

8.6 Implementation Patterns Cross-Reference

The local Python loader (`ipi_bronze_load.py`) connects to the same Azure SQL Database from outside Fabric, using `pyodbc` + `azure-identity`. The connection string requirements, token packing for `pyodbc`, auto-pause handling, and bulk insert performance findings are documented as portfolio-grade content in **Section 9: Implementation Patterns**.

9. Implementation Patterns: Azure SQL Connection

This section documents real-world findings from the implementation of the IPI loader (`ipi_bronze_load.py`) during Phase 4.5 (Local Python Validation). It covers five technical patterns and a complete authentication strategy.

***Why this section exists** — these findings are not present in Microsoft's official documentation in any consolidated form. They were surfaced through hands-on implementation against a real Azure SQL Database. Documenting them here serves three purposes: (a) it preserves the engineering knowledge for the project's reproducibility, (b) it demonstrates senior-level technical depth for portfolio review, (c) it accelerates anyone reproducing the project by avoiding the same time-consuming discovery process.*

9.1 Connection String Requirements (Entra-Only Auth)

The connection string for `pyodbc` with Entra-only authentication has four non-obvious requirements. Missing any of them produces failure modes that are difficult to diagnose:

Parameter	Required value	Failure mode if wrong
Encrypt	yes	HYT00 timeout during the Entra ID handshake; ODBC driver fails silently without a clear error
TrustServerCertificate	no	Required for production safety; certificate validation is enforced (no MITM tolerance)
Connection Timeout	≥ 60 seconds	Serverless auto-pause cold-start takes ~30–60 seconds; lower timeout fails on first connection after pause
SERVER format	host,port (no 'tcp:' prefix)	Driver adds 'tcp:' internally; manually adding it produces double-prefix and connection failure

Working connection string template:

```
DRIVER={ODBC Driver 18 for SQL Server};  
SERVER=spi-sqlserver-gb.database.windows.net,1433;  
DATABASE=spi_staging;  
Encrypt=yes;  
TrustServerCertificate=no;  
Connection Timeout=60;
```

Note — no UID or PWD in the connection string. The Entra ID token is passed via `attrs_before`, not via the connection string. See Section 9.2.

9.2 Token Packing for pyodbc

This is the most error-prone part of the integration and the most poorly documented. Entra ID access tokens cannot be passed as plain strings to `pyodbc`. They must be packed into a binary format and passed via the ODBC `attrs_before` parameter.

The packing format is:

- 4-byte little-endian integer = length of the encoded token in bytes
- Followed by: the access token encoded as UTF-16 little-endian
- Passed via `attrs_before={1256: token_bytes}`
- The constant 1256 is `SQL_COPT_SS_ACCESS_TOKEN`, the ODBC connection attribute for Entra access tokens

Working token packing pattern:

```
import struct
from azure.identity import DefaultAzureCredential
import pyodbc

# 1. Acquire token via Azure Identity
credential = DefaultAzureCredential()
token = credential.get_token("https://database.windows.net/.default")

# 2. Pack token into ODBC binary format
token_bytes = token.token.encode("utf-16-le")
packed = struct.pack(">i", len(token_bytes)) + token_bytes

# 3. Pass via attrs_before
SQL_COPT_SS_ACCESS_TOKEN = 1256
conn = pyodbc.connect(connection_string,
                      attrs_before={SQL_COPT_SS_ACCESS_TOKEN: packed})
```

Common pitfalls:

- Using UTF-8 instead of UTF-16-LE: produces an obscure 'Invalid token' error from the SQL Server.
- Using big-endian struct format: '>i' instead of '=i' produces invalid length prefix on most platforms.
- Passing the raw token string in the connection string with AccessToken=: this syntax is for ADO.NET, not ODBC; pyodbc ignores it.
- Forgetting .default at the end of the resource scope: produces a token without the right audience claim.

9.3 Auto-Pause Handling

Azure SQL Serverless databases auto-pause after ~1 hour of inactivity to save cost. The first connection attempt after a pause raises a specific error that must be handled with retry logic, while other errors should fail immediately.

Aspect	Detail
Error code	40613 (database is currently unavailable)
Error message	'Cannot open database "... " requested by the login. The database is currently resuming...'
Resume time	Typically 30–60 seconds; can occasionally extend to 90 seconds
Recommended retry	3 attempts with 30-second wait between attempts
Other errors	Do NOT retry. Wrong credentials, missing permissions, or schema errors are structural and retry-resistant

Resume-aware retry pattern:

```

import time
import pyodbc

MAX_ATTEMPTS = 3
RESUME_WAIT_SECONDS = 30

def connect_with_resume_retry(conn_str, attrs):
    last_error = None
    for attempt in range(1, MAX_ATTEMPTS + 1):
        try:
            return pyodbc.connect(conn_str, attrs_before=attrs)
        except pyodbc.OperationalError as e:
            if "40613" in str(e) and attempt < MAX_ATTEMPTS:
                print(f"DB resuming (attempt {attempt}); waiting {RESUME_WAIT_SECONDS}s...")
                time.sleep(RESUME_WAIT_SECONDS)
                last_error = e
                continue
            raise
    raise last_error

```

Operational note — the first script run after a pause typically takes ~45 seconds to establish a usable connection. Subsequent runs within the active window connect in ~1 second. Plan demos accordingly: warm up the database with a trivial query 60 seconds before a live demonstration.

9.4 Bulk Insert Performance

This finding produced the most dramatic improvement of the entire Phase 4.5 work: a **~28x speedup** in IPI ingestion, plus avoidance of session-timeout-induced failures.

Approach	Time for 309K rows	Notes
Row-by-row INSERT	~14 minutes	Default pyodbc behavior; one round-trip per row; hits Azure SQL session timeout (~10 min on open transactions)
fast_executemany + batch 5000 + commit per batch	~30 seconds	~28x improvement; transactions stay short; predictable performance

The three optimization knobs:

- `cursor.fast_executemany = True`: enables ODBC parameter array binding. The driver sends batches of rows in a single network round-trip instead of one per row. This alone is responsible for most of the speedup.
- **Batch size of 5000 rows per executemany call**: balances memory usage on the client with round-trip efficiency. Smaller batches lose the round-trip benefit; larger batches risk hitting parameter limits.
- **Commit per batch (not per row, not at end)**: keeps each transaction short. Long open transactions on Azure SQL Serverless can exceed the implicit 10-minute limit and fail the entire load.

Working bulk insert pattern:

```
import pyodbc

BATCH_SIZE = 5000

def bulk_insert(conn, rows, table_name):
    cursor = conn.cursor()
    cursor.fast_executemany = True

    placeholders = ",".join(["?"] * len(rows[0]))
    sql = f"INSERT INTO {table_name} VALUES ({placeholders})"

    for i in range(0, len(rows), BATCH_SIZE):
        batch = rows[i : i + BATCH_SIZE]
        cursor.executemany(sql, batch)
        conn.commit() # commit per batch
        print(f"Inserted batch {i // BATCH_SIZE + 1} ({len(batch)} rows)")
```

9.5 ODBC Driver Setup for macOS ARM64 (M3)

Apple Silicon Macs require a specific Homebrew tap for the Microsoft ODBC driver. Standard installation methods (pip, generic Homebrew) install the wrong architecture or an outdated driver version.

Required setup commands:

```
# 1. Add the Microsoft mssql-release tap (one-time)
brew tap microsoft/mssql-release \
  https://github.com/Microsoft/homebrew-mssql-release

# 2. Install ODBC Driver 18 (NOT 17 - Driver 18 is required for modern Entra auth)
brew install msodbcsql18

# 3. Optional: install command-line tools (sqlcmd, bcp)
brew install mssql-tools18

# 4. Validate installation
odbcinst -q -d
# Expected output: [ODBC Driver 18 for SQL Server]
```

Common pitfalls on Apple Silicon:

- Installing ODBC Driver 17 instead of 18: Driver 17 lacks support for modern Entra ID token authentication; connections silently fail at the handshake.
- Using `pip install pyodbc` without first installing the ODBC driver: `pyodbc` compiles but cannot find a usable driver at runtime.
- Mixing Intel and ARM64 Homebrew installations (when running under Rosetta): yields a driver in the wrong architecture; reinstall under the native ARM64 Homebrew prefix (`/opt/homebrew`).

9.6 Authentication Strategy — Three Scenarios

The choice of credential class from `azure-identity` depends on the execution context. Documenting all three scenarios (rather than only the one used) demonstrates architectural maturity: a freelance reviewer expects an engineer to understand *when* to use each tool, not just *which one works*.

Scenario	Auth Method	Rationale
One-time / occasional load	InteractiveBrowserCredential	Maximum security; MFA challenge on every run; no token caching. Appropriate for ad-hoc historical loads or sensitive operations.

Scenario	Auth Method	Rationale
Iterative local development	DefaultAzureCredential (with az login cached)	No browser popup between runs. Token cached by Azure CLI for ~12–24 hours. The right tool for active development sessions where the same script runs dozens of times.
Production / Fabric pipelines	Service Principal or Managed Identity	No human interaction. Suitable for scheduled execution. Service Principal credentials live in a key vault; Managed Identity is preferred when available (no secrets to rotate).

Implementation choice for this project

The IPI loader uses **Scenario 2: DefaultAzureCredential**. The reasoning:

- The loader is run iteratively during Phase 4.5 development — dozens of executions over a few days.
- DefaultAzureCredential automatically tries multiple credential sources in order: environment variables, managed identity, Azure CLI, VS Code, etc. The Azure CLI cache (set up via `az login`) is hit transparently.
- The pattern matches what would be used in a real client engagement during the development phase: developers authenticate once via `az login`, the script reuses the cached token, and there is no browser interruption during the dev loop.
- Migration to Service Principal for production-style execution requires only changing the credential class — the rest of the loader code remains unchanged.

Working DefaultAzureCredential pattern:

```
from azure.identity import DefaultAzureCredential

# DefaultAzureCredential will try, in order:
# 1. EnvironmentCredential (env vars)
# 2. ManagedIdentityCredential (in Azure)
# 3. SharedTokenCacheCredential (VS Code, etc.)
# 4. AzureCliCredential (az login cache)
# 5. AzurePowerShellCredential
# 6. InteractiveBrowserCredential (last resort)

credential = DefaultAzureCredential()
token = credential.get_token("https://database.windows.net/.default")
# token.token is the access token string (not yet packed for pyodbc)
```

Why documenting all three is differential portfolio content

Most candidates document only the authentication method they used. Documenting all three with their respective use cases, plus the rationale for the chosen one, demonstrates architect-level thinking: the engineer has considered alternatives, evaluated tradeoffs, and made a deliberate choice. This is the kind of section that separates senior portfolios from tutorial-level ones.

10. Logging Framework Implementation

All notebook and pipeline executions write to a single central log table in the Lakehouse. This section specifies the table schema and the helper function pattern that notebooks and pipelines use to write to it.

10.1 Log Table DDL

The logging table lives in the Lakehouse (not the Warehouse) because it is operational metadata, not analytical data.

```
-- Created once via a Lakehouse notebook; executed as Spark SQL
CREATE TABLE spi_log_pipeline_execution (
  run_id          STRING,
  pipeline_name   STRING,
  layer           STRING,
  source          STRING,
  status          STRING,
  start_time      TIMESTAMP,
  end_time        TIMESTAMP,
  error_message   STRING,
  rows_processed  BIGINT
) USING DELTA;
```

Column	Meaning	Example
run_id	GUID generated by master pipeline, propagated to all children	a3f2...
pipeline_name	Pipeline or notebook identifier	spi_nb_bronze_energy
layer	Medallion layer	bronze
source	Source identifier or 'all'	energy
status	Current state	running, success, failed
start_time	UTC timestamp of start	2026-04-10 14:30:00
end_time	UTC timestamp of completion (null while running)	2026-04-10 14:32:15
error_message	Exception or DQ warning summary	DQ-08: 3 negative values in IPC
rows_processed	Rows written by this execution	45230

10.2 Helper Function Pattern

A shared notebook (or %run-included module) exposes three functions used by every other notebook:

Function	Purpose	Call timing
log_start(run_id, pipeline_name, layer, source)	Inserts a new row with status = 'running', start_time = now	First line of main block
log_end_success(run_id, pipeline_name, rows_processed)	Updates the row with status = 'success', end_time = now, rows_processed	Last line of main block (success path)
log_end_failure(run_id, pipeline_name, error_message)	Updates the row with status = 'failed', end_time = now, error_message	Inside exception handler

Notebook integration pattern:

```
run_id = <injected by pipeline parameter>
pipeline_name = 'spi_nb_bronze_energy'

log_start(run_id, pipeline_name, 'bronze', 'energy')
try:
    rows = main()
    log_end_success(run_id, pipeline_name, rows)
except Exception as e:
    log_end_failure(run_id, pipeline_name, str(e))
    raise
```

10.3 Pipeline Integration

Data Pipelines write their own log rows at the pipeline level (master start, layer start, layer end) via a small 'Stored procedure' or 'Script' activity. The `run_id` generated by the master pipeline is passed as a parameter to all child pipelines and notebooks.

10.4 Querying the Log

Typical diagnostic queries:

```
-- Last run summary
SELECT pipeline_name, layer, source, status,
       start_time, end_time, rows_processed
FROM spi_log_pipeline_execution
WHERE run_id = (SELECT MAX(run_id) FROM spi_log_pipeline_execution)
ORDER BY start_time;

-- Recent failures across all runs
SELECT pipeline_name, source, start_time, error_message
FROM spi_log_pipeline_execution
WHERE status = 'failed'
ORDER BY start_time DESC
LIMIT 20;
```


11. Power BI Technical Specification

The Power BI deliverable is a PBIP-format semantic model and report, connected to `spi_warehouse` via Direct Lake. This section specifies the technical configuration. The report's functional design (pages, visuals) is defined in Phase 3 Section 7 and is not repeated here.

11.1 Connectivity

Direct Lake is the chosen connectivity mode. Direct Lake loads data from the Warehouse's underlying Delta storage into VertiPaq on demand, without import refresh and without DirectQuery latency. It is the modern default for Fabric-native Power BI.

Requirements for Direct Lake. The Warehouse tables must use Delta storage (default in Fabric Warehouse), must not exceed workspace capacity limits, and must be in the same workspace as the semantic model. All these conditions are met by the DEV and PROD workspace setup.

11.2 Semantic Model — Tables

Model table	Source table
Fact Indicators	<code>spi_warehouse.dbo.spi_fact_indicators</code>
Indicator	<code>spi_warehouse.dbo.spi_dim_indicator</code>
Region	<code>spi_warehouse.dbo.spi_dim_region</code>
Calendar	<code>spi_warehouse.dbo.spi_dim_calendar</code>
Measure Type	<code>spi_warehouse.dbo.spi_dim_measure</code>
Source	<code>spi_warehouse.dbo.spi_dim_source</code>

Display names in the model use friendly casing (title case with spaces). Physical column names in the database remain snake_case.

11.3 Relationships

From (many)	Column	To (one)	Column	Cardinality	Cross-filter
Fact Indicators	<code>indicator_key</code>	Indicator	<code>indicator_key</code>	Many-to-one	Single
Fact Indicators	<code>region_key</code>	Region	<code>region_key</code>	Many-to-one	Single
Fact Indicators	<code>calendar_key</code>	Calendar	<code>calendar_key</code>	Many-to-one	Single
Fact Indicators	<code>measure_key</code>	Measure Type	<code>measure_key</code>	Many-to-one	Single
Fact Indicators	<code>source_key</code>	Source	<code>source_key</code>	Many-to-one	Single

All relationships are **single-direction**. Bi-directional filters are not used.

11.4 Calendar Dimension Marking

The calendar table is marked as the date table, even though it has monthly (not daily) granularity. `calendar_key` (YYYYMM integer) is used as the mark.

11.5 DAX Measures

11.5.1 Indicator Value

```
Indicator Value =
SUM ( 'Fact Indicators'[indicator_value] )
```

Returns the value filtered by the current context. Used in all cards, charts, and tables.

11.5.2 Latest Period

```
Latest Period =
CALCULATE (
    MAX ( 'Calendar'[year_month] ),
    'Fact Indicators'[indicator_value] <> BLANK ()
)
```

Returns the most recent year_month for which data exists.

11.5.3 Latest Value

```
Latest Value =
CALCULATE (
    [Indicator Value],
    FILTER (
        ALL ( 'Calendar' ),
        'Calendar'[year_month] = [Latest Period]
    )
)
```

Returns the indicator value for the latest available period. Used in Overview KPI cards.

11.5.4 Trend Direction

```
Trend Direction =
VAR CurrentValue = [Latest Value]
VAR PreviousValue =
    CALCULATE (
        [Indicator Value],
        FILTER (
            ALL ( 'Calendar' ),
            'Calendar'[calendar_key] = MAX ( 'Calendar'[calendar_key] ) - 1
        )
    )
RETURN
    SWITCH (
        TRUE (),
        CurrentValue > PreviousValue, "Up",
        CurrentValue < PreviousValue, "Down",
        "Flat"
    )
```

Returns a string indicating trend direction vs. previous period. Used for trend arrows in KPI cards.

*Note on derived measures (YoY, MoM, YTD, QTD) — these are **not** computed as DAX measures; they are precomputed rows in the fact table (Phase 3 Section 5). The Power BI user selects the desired measure type via a slicer on the Measure Type dimension.*

11.6 Report File Format — PBIP

The report is saved in **PBIP (Power BI Project)** format rather than PBIX. PBIP decomposes the semantic model into TMDL files and the report into JSON, enabling Git version control with meaningful diffs.

```
/src/power-bi/
■■■■ spi_spain_indicators.Report/
■■■■ spi_spain_indicators.SemanticModel/
■■■■ .gitignore
■■■■ spi_spain_indicators.pbip
```

12. Deployment Strategy

The project is developed in DEV and promoted to PROD near the end of Phase 5. This section specifies the promotion mechanism.

12.1 Fabric Deployment Pipelines

Fabric Deployment Pipelines provide a native mechanism to promote artifacts between workspaces. The pipeline binds the DEV workspace as Stage 1 and the PROD workspace as Stage 2.

Stage	Workspace
Development	spi-spain-indicators-dev
Production	spi-spain-indicators-prod

12.2 What Gets Promoted

Artifact type	Promoted?
Lakehouse (empty shell)	Yes
Warehouse (empty shell with DDL)	Yes
Notebooks	Yes
Pipelines	Yes
Dataflows Gen2	Yes
Semantic Model	Yes
Power BI Report	Yes
Data (Bronze, Silver, Gold rows)	No — PROD regenerates its own data

12.3 Deployment Rules

Parameter	DEV value	PROD value
environment	dev	prod
data_scope	last_24_periods	full_history
Azure SQL connection	conn_azure_sql_ipi_source_dev	conn_azure_sql_ipi_source_prod

12.4 Deployment Sequence

1. DEV workspace is stable; all pipelines run end-to-end successfully.
2. PROD workspace is created and bound to the Deployment Pipeline as Stage 2.
3. Deployment rules are configured in Fabric's Deployment Pipelines UI.
4. 'Deploy' is triggered from DEV → PROD. All artifacts are copied.
5. The master pipeline in PROD is triggered manually with `data_scope = full_history`.
6. Screenshots are captured in PROD for the portfolio. DEV remains as the active editing surface.
7. Before day 53 of the trial, PROD is migrated to F2 PAYG capacity (paused by default) per the Phase 0 strategy.

13. Testing and Validation Plan

Validation is performed per layer using a combination of row-count reconciliation, sample inspection, and data quality rule verification.

13.1 Bronze Validation

Check	Query pattern
Row count per source matches expected volume	SELECT COUNT(*) FROM
No rows missing the ingestion timestamp	SELECT COUNT(*) FROM WHERE _ingestion_timestamp IS NULL
Source file / table identifier populated	SELECT DISTINCT _source_file FROM
Sample rows match source file content	Visual comparison of 10 rows vs. raw file

13.2 Silver Validation

Check	Query pattern
Row count reduced from Bronze in an expected way	Compare COUNT(*) before/after; justify the difference
Geographic filter applied: only three regions	SELECT DISTINCT region_name FROM must return exactly the three canonical values
Year and month parsed correctly	SELECT MIN(year), MAX(year), MIN(month), MAX(month)
No null indicator_value where business rules require non-null	Check DQ-03 per source
No duplicates on natural key	GROUP BY ... HAVING COUNT(*) > 1 returns zero

13.3 Gold Validation

Check	Query pattern
Every fact row has valid FKs	Left join to each dimension; count nulls
Composite PK is unique	GROUP BY all five keys, HAVING COUNT(*) > 1 returns zero
Five measure types present per indicator-region-period	SELECT DISTINCT measure_key should return 5 for a sample combination
Derived measure nulls conform to business rules	Inspect NULL patterns per Phase 3 Section 5
Row count roughly equals (Silver IDX rows) × 5	Rough sanity check of derived measure expansion

13.4 End-to-End Validation

Check	Approach
Power BI cards show correct values	Compare Power BI card value to equivalent SQL query against Warehouse
Slicer changes propagate correctly	Change region slicer; verify card updates
Measure Type slicer switches measures correctly	Toggle IDX → YoY → MoM; verify chart values change appropriately

13.5 Pre-PROD Checklist

- All notebooks complete successfully with `full_history` scope.
- All pipelines in the master flow complete with success status.
- Log table has one row per layer-source combination per run, with no orphaned running statuses.
- Gold fact row count matches expected volume (approximately 300,000 rows).
- All Power BI pages render without errors.
- DAX measures return non-blank values for realistic slicer selections.

14. Troubleshooting Guide

This section anticipates errors that are likely to appear during Phase 5, based on the characteristics of each source and the Fabric components involved. Additional troubleshooting entries are added during Phase 5 as real errors are encountered.

Living section — this is the starting set. Phase 5 will expand it with hands-on findings. The expanded version is versioned in the GitHub repository (Lessons-Learned.md).

14.1 Encoding — ISO-8859-15 CSV Files

Symptom. Spanish characters (ñ, á, é, ü) appear garbled in Bronze or Silver tables.

Cause. The INE CSV is encoded in ISO-8859-15; the ingestion code is reading it as UTF-8.

Resolution. In the Copy Activity or notebook, set `encoding = 'iso-8859-15'` explicitly at read time. Write to Bronze as UTF-8.

14.2 Merged Cells — XLS Legacy

Symptom. The construction XLS files parse into DataFrames with NaN values where merged cells should propagate.

Cause. `xlrd` does not automatically fill merged cells.

Resolution. Detect merged ranges via `sheet.merged_cells` and forward-fill the value across the range.

14.3 API Timeout — REE Energy

Symptom. HTTP 504 or connection reset when fetching a full year of monthly data.

Resolution. The notebook retry logic handles this automatically. If all retries fail, split the request into smaller ranges (e.g., 6-month windows).

14.4 Direct Lake — Fallback to DirectQuery

Symptom. Power BI visuals show slower response than expected; the semantic model indicates 'DirectQuery fallback' has occurred.

Resolution. Reduce the cardinality of slicers, or simplify overly complex DAX measures. For the portfolio, occasional fallback is acceptable.

14.5 Azure SQL Auto-Pause Cold Start

Symptom. First connection to Azure SQL after ~1 hour of inactivity returns error 40613 ('database resuming').

Resolution. Apply the resume-aware retry pattern documented in Section 9.3. For demos, warm up the database 60 seconds before by issuing a trivial query.

14.6 Dataflow Gen2 — Destination Not Writing

Symptom. A Dataflow Gen2 runs successfully but the destination table appears empty or outdated.

Resolution. Verify destination configuration uses 'Replace' (truncate and reload) for Silver and Gold layers. Re-publish the dataflow. Execute from the pipeline, not the preview pane.

14.7 Trial Capacity Exhaustion

Symptom. Pipelines queue indefinitely or fail with 'capacity limit reached' errors.

Resolution. Wait 15-30 minutes and retry. Reduce concurrency by running pipelines sequentially. If persistent, verify the trial is still active.

15. Next Steps

With the technical specification formalized, the project proceeds to **Phase 5: Development in Fabric**, which will deliver:

- Activation of the Fabric Trial Capacity and creation of the DEV workspace.
- Implementation of all Bronze, Silver, and Gold components per the specifications in this document.
- Power BI semantic model and report in PBIP format.
- Export of all artifacts from Fabric to the local repository structure.
- Capture of curated screenshots evidencing the working solution in PROD.
- Loom demo video walking through the solution end-to-end.
- Migration of PROD to F2 PAYG capacity (paused by default) before day 53 of the trial.

Phase Progress

0	Context & Strategy Deliverable: Project Brief (PDF)	COMPLETE
1	Requirements & Source Catalog Deliverable: Requirements Document + Source Catalog	COMPLETE
2	Architecture Design Deliverable: Architecture Design Document	COMPLETE
3	Functional Specification Deliverable: Functional Specification (PDF)	COMPLETE
4	Technical Specification Deliverable: Technical Specification (PDF)	COMPLETE
5	Development in Fabric Deliverable: Working Solution + Code	NEXT
6	GitHub Publication Deliverable: GitHub Repository + README	PENDING

16. Changelog

This document is now considered a frozen deliverable. Future findings during Phase 5 (Fabric development) will be captured in the GitHub repository (README, lessons-learned.md, and per-component READMEs), not by re-versioning this PDF.

Version 1.1 — May 2026

Pre-development refresh after Phase 4.5 (Local Python Validation). Five substantive content additions documenting real environment values, the local development stack, the Python package stack, and — most significantly — a complete new section on Azure SQL connection patterns and authentication strategy surfaced during the IPI loader implementation.

Area	Change	Section
Author role	Updated from 'Senior Data Analyst' to 'Analytics Engineer' to reflect the target professional positioning	Cover
Environment Reference	New subsection documenting the actual provisioned values: tenant ID, primary domain, identities, Azure subscription, Azure SQL server/database, GitHub repository	Section 2.6
Local Development Stack	Section 2.4 expanded from generic 'Development Tooling' to the actual stack used: macOS Tahoe 26.4 on MacBook Air M3, Python 3.12.13, VS Code 1.118 with MSSQL extension, Azure CLI for credential caching	Section 2.4
Python Package Stack	New subsection documenting the project's Python dependencies with versions: pandas, openpyxl, xlrd, requests, pyarrow, pyodbc, azure-identity	Section 3.7
Implementation Patterns: Azure SQL	New section (most significant addition) covering five real-world findings from the IPI loader: connection string requirements (Encrypt=yes, TrustServerCertificate, Connection Timeout, SERVER format), token packing for pyodbc with the SQL_COPT_SS_ACCESS_TOKEN binary format, auto-pause handling with retry on error 40613, bulk insert performance (~28x speedup with fast_executemany + batching), ODBC ARM64 driver setup for Apple Silicon	Section 9 (new)
Authentication Strategy	New subsection documenting three authentication scenarios (InteractiveBrowserCredential, DefaultAzureCredential, Service Principal/Managed Identity) with rationale for each and the implementation choice for this project	Section 9.6
Renumbering	Sections shifted by +1 to accommodate the new Section 9: Logging (9→10), Power BI (10→11), Deployment (11→12), Testing (12→13), Troubleshooting (13→14), Next Steps (14→15)	Sections 10–15
Frozen status	Document marked as frozen deliverable. Subsequent findings during Phase 5 will be captured in the GitHub repository, not via additional PDF versions	Section 16

Version 1.0 — April 2026

Initial Phase 4 deliverable. Defined environment provisioning, code conventions and standards, complete Gold layer DDL (1 fact + 5 dimensions), notebook structural specifications, Dataflow Gen2 step designs, pipeline orchestration design, connections and authentication architecture, logging framework, Power BI technical specification with DAX measures, deployment strategy, testing plan, and troubleshooting guide.

Document version 1.1 | May 2026 | Phase 4 deliverable (frozen)